

Automação de testes na prática

A configuração de um **ambiente de automação de testes** é uma etapa crítica para garantir que os testes sejam executados corretamente e de forma eficiente. Um ambiente bem configurado pode ajudar a otimizar o processo de **testes automatizados** e garantir que os resultados sejam confiáveis.

Alguns passos para **configurar o ambiente** de automação de testes, com foco nas etapas principais, ferramentas e boas práticas são:

1. Escolher as Ferramentas de Automação

· A primeira etapa para configurar seu ambiente de testes é escolher as ferramentas certas com base nas necessidades do projeto. Isso inclui decidir quais ferramentas usar para testes de UI, APIs, performance e segurança. Para exemplificar, vamos usar as ferramentas mencionadas:

- Selenium ou Cypress (para testes web).
- Appium ou Espresso (para testes móveis).
- Jenkins ou GitLab CI (para integração contínua).
- JMeter ou Gatling (para testes de performance).

2. Configuração do Ambiente de Desenvolvimento

· Instalação das Ferramentas Básicas

- Java (se necessário, para ferramentas como Selenium ou Appium):

1. Baixe e instale o JDK (Java Development Kit).

2. Configure as variáveis de ambiente JAVA_HOME e PATH.

- Node.js e NPM (se necessário, para ferramentas como Cypress ou Playwright):

1. Baixe e instale o Node.js.

2. Verifique se o NPM (Node Package Manager) está funcionando com o comando `npm -v`.

- Maven ou Gradle (caso utilize Java e queira uma ferramenta de build):

1. Para Maven: Instale o Maven.

2. Para Gradle: Instale o Gradle.

3. Ferramentas de Automação e Dependências

· Configuração do Selenium (Exemplo)

- Baixe o WebDriver:

1. Para Chrome: Baixe o ChromeDriver.

2. Para Firefox: Baixe o GeckoDriver.

3. Para Edge: Baixe o EdgeDriver.

- Instalar dependências no Maven (se usando Java)

1. <dependency>

2. <groupId>org.seleniumhq.selenium</groupId>

3. <artifactId>selenium-java</artifactId>

4. <version>3.141.59</version>

5. </dependency>

- Baixar bibliotecas para Cypress:

1. npm install cypress --save-dev

4. Configuração do Ambiente de Testes

- Configuração de Navegadores e Dispositivos

- Navegadores: Certifique-se de que o WebDriver correto esteja instalado e configurado para os navegadores que você deseja testar.

- Emuladores/Dispositivos Móveis: Caso esteja testando em dispositivos móveis, configure os emuladores ou dispositivos reais com as ferramentas apropriadas (como Android Emulator ou iOS Simulator).

- Configurando o Selenium Grid (Opcional)

- Se você precisar testar em vários navegadores e plataformas, pode configurar o Selenium Grid para distribuir os testes de maneira paralela:

1. Configuração do Hub:

- a. java -jar selenium-server-standalone.jar -role hub

2. Configuração dos Nodes:

- a. java -jar selenium-server-standalone.jar -role node -
hub <http://localhost:4444/grid/register>

5. Configuração do Ambiente de Integração Contínua (CI)

- Instalar Jenkins

- Baixe o Jenkins: Baixe o Jenkins.

- Configuração inicial:

1. Após a instalação, acesse <http://localhost:8080> para configurar o Jenkins.

2. Instale os plugins necessários, como Git, Maven, Selenium, etc.

3. Configure um pipeline de integração contínua para rodar os testes automaticamente.

- Configuração do GitLab CI

- Configurar o repositório GitLab:
- Crie um arquivo .gitlab-ci.yml na raiz do seu projeto:

1. stages:

2. - test

3. test:

4. script:

5. - npm install

6. - npm test

6. Configuração do Ambiente de Testes de Performance

· Configuração do Apache JMeter

- Baixe o JMeter.

- Execute o JMeter:

1. Execute o JMeter para testar a carga e performance da sua aplicação.

2. Configure testes de carga para APIs ou páginas web no JMeter, e crie relatórios de desempenho.

· Configuração do Gatling

- Baixar e configurar o Gatling: Baixe o Gatling.

- Criar cenários de teste:

1. Gatling utiliza Scala para definir cenários de testes de carga, como exemplo:

a. setUp(

b. scn.inject(atOnceUsers(1))

c.).protocols(http.baseUrl("https://yourapp.com"))

7. Boas Práticas para Configuração do Ambiente

- Automatização de Configuração: Utilize Docker ou Vagrant para criar ambientes consistentes e facilmente replicáveis.

- Versionamento: Mantenha as ferramentas e dependências versionadas para garantir que todos os desenvolvedores e testers usem as mesmas versões.

- Ambiente de Produção e Teste Separados: Mantenha ambientes isolados para testes e produção. Use containers para simular diferentes ambientes (ex: Docker).

- Integração com CI/CD: Garanta que seus testes estejam integrados a pipelines de CI/CD para que sejam executados automaticamente em cada commit ou deploy.

A estruturação de testes automatizados é uma etapa crucial para garantir que seus testes sejam manuteníveis, reutilizáveis e eficazes. Uma boa estrutura

facilita a expansão e modificação dos testes à medida que o projeto evolui. Abaixo, vamos explorar como estruturar seus testes de maneira eficiente, desde a organização do código até as boas práticas.

Estrutura do Projeto de Testes

Diretórios e Organização de Arquivos

Uma estrutura de diretórios bem-organizada facilita a manutenção e expansão dos testes. Aqui está um exemplo básico de estrutura de diretórios para um projeto de testes automatizados com Selenium e Java:

```
/project-root
/src
/main
/java
/com
/yourcompany
/app
(arquivos da aplicação)
/test
/java
/com
/yourcompany
/tests
/functional
(testes de funcionalidade)
/integration
(testes de integração)
/smoke
(testes de verificação rápida)
/regression
(testes de regressão)
/utils
(código utilitário)
TestBase.java
TestRunner.java
TestUtils.java
```

Classificação de Testes

1. Testes Funcionais: Testam funcionalidades específicas do sistema.

2. Testes de Integração: Verificam como os diferentes componentes do sistema interagem.

3. Testes de Regressão: Certificam-se de que o sistema não quebrou após modificações.

4. Testes de Smoke: Verificações rápidas para garantir que a aplicação não tenha falhas críticas após mudanças.

Padrão de Nomenclatura

Use nomes consistentes e claros para os arquivos de teste e classes de testes.

Exemplo:

- LoginTest.java
- UserRegistrationTest.java
- APIAuthenticationTest.java

Padrões de Codificação e Boas Práticas

Separação de Preocupações

É importante manter separados os componentes de automação e lógica de negócios. Isso facilita a manutenção e melhora a leitura do código.

- Page Object Model (POM): Organize os testes de interface utilizando o padrão Page Object Model. Esse padrão ajuda a separar a lógica de interação com a UI da lógica de testes, tornando os testes mais manuteníveis.
- Testes Unitários e Funcionais Separados: Crie classes separadas para testes unitários (isolados) e testes funcionais (que interagem com a interface do usuário).

Testes Reutilizáveis

Mantenha funções e métodos reutilizáveis, como:

- Configurações de inicialização (Ex: configuração de WebDriver).
- Métodos utilitários (Ex: manipulação de dados de teste).
- Métodos de validação (Ex: assertivas de valores).

Ferramentas de Suporte à Estruturação

Frameworks de Teste

Use frameworks de teste que forneçam estruturas de execução e relatórios para organizar a execução dos testes.

- JUnit / TestNG (para Java): São frameworks que permitem organizar e executar testes, fornecer relatórios e gerenciar dependências.

- Cucumber (para BDD): Para quem adota Behavior Driven Development (BDD), o Cucumber permite escrever testes em linguagem natural (Gherkin), facilitando a colaboração entre desenvolvedores e equipes de negócios.

Relatórios e Logs

- Allure Report: Para gerar relatórios bonitos e compreensíveis.
- ExtentReports: Oferece uma interface gráfica para relatórios de teste.
- Log4j ou SLF4J: Ferramentas de log para registrar informações durante a execução dos testes.

Boas Práticas de Estruturação de Testes Automatizados

- Reusabilidade e Modularidade: Crie métodos e classes que podem ser reutilizados, como funções de login, registros e navegação.
- Ambiente de Teste Isolado: Utilize ambientes isolados para garantir que os testes não afetem a base de dados ou sistemas externos.
- Manutenção Regular: Revise e atualize os testes regularmente para acompanhar mudanças no sistema.
- Dados de Teste Dinâmicos: Sempre que possível, use dados dinâmicos para testes (ex: com mocking ou stubbing de APIs).
- Execução Paralela: Organize seus testes para execução paralela em diferentes ambientes, garantindo economia de tempo.
- Cobertura de Testes: Certifique-se de que seus testes cubram as funcionalidades mais críticas, evitando redundâncias.

O **desenvolvimento de testes automatizados** envolve a criação e implementação de testes que verificam automaticamente o comportamento de uma aplicação, garantindo que ela funcione conforme esperado. Esse processo é fundamental para aumentar a eficiência, reduzir erros humanos e permitir que as equipes se concentrem em partes mais complexas do software.

Aqui estão as principais etapas e práticas envolvidas no desenvolvimento de testes automatizados:

1. Planejamento e Definição de Objetivos

Antes de começar a escrever qualquer teste, é essencial planejar qual será o **escopo** dos testes automatizados. Perguntas importantes incluem:

- **Qual funcionalidade será testada?**
- **Quais tipos de testes serão automatizados?** (funcionais, de integração, de UI,

etc.)

- **Quais são os critérios de aceitação?**
- **Quais ferramentas serão utilizadas?**

O planejamento ajuda a garantir que você automatize as partes certas do aplicativo e que os testes cubram todos os requisitos essenciais.

2. Escolha das Ferramentas de Automação

Escolher as ferramentas de automação certas depende do tipo de aplicação e da stack utilizada. Algumas ferramentas populares incluem:

Ferramentas de Automação para Testes de UI

- **Selenium** (Java, C#, Python): Para automação de **testes de navegador**.
- **Cypress** (JavaScript): Para **testes de front-end** em aplicações modernas.
- **Playwright** (JavaScript/TypeScript): Para automação de testes em **navegadores modernos e dispositivos móveis**.
- **Appium**: Para automação de **testes em dispositivos móveis**.

Ferramentas para Testes de API

- **Postman**: Para testes de **APIs REST**.
- **RestAssured** (Java): Para automação de **testes de API REST**.
- **SoapUI**: Para **testes de APIs SOAP**.

Frameworks de Teste

- **JUnit** ou **TestNG** (Java): Frameworks populares para testes unitários.
- **Mocha** ou **Jest** (JavaScript): Para testes unitários em aplicações **Node.js**.

3. Escrever Casos de Teste Automatizados

Estrutura de um Caso de Teste Automatizado

Um **caso de teste automatizado** pode ser estruturado da seguinte forma:

1. **Preparação**: Configuração do ambiente necessário para o teste.
2. **Ação**: Execução da funcionalidade que será testada.
3. **Verificação**: Verificação de que o resultado esperado foi obtido.
4. **Limpeza**: Restauração do sistema para o estado original, se necessário.

4. Testes Reutilizáveis e Estruturas de Código

Ao escrever testes automatizados, é fundamental manter o código limpo, modular e reutilizável. Algumas práticas importantes incluem:

Page Object Model (POM)

O **Page Object Model (POM)** é um padrão que ajuda a separar a lógica de teste da lógica de interação com a interface do usuário, tornando o código mais organizado e fácil de manter.

5. Integração Contínua (CI) para Testes Automatizados

Após escrever os testes automatizados, é essencial integrá-los ao **pipeline de integração contínua (CI)**. Isso garantirá que os testes sejam executados automaticamente sempre que houver alterações no código.

6. Relatórios e Logs

Gerar **relatórios** de execução de testes e **logs** detalhados é fundamental para acompanhar o progresso e os resultados dos testes.

- **Allure Report**: Para gerar relatórios bonitos e compreensíveis.
- **ExtentReports**: Para uma visualização mais rica dos testes.
- **Log4j** ou **SLF4J**: Para registrar logs detalhados durante a execução dos testes, o que ajuda na depuração.

7. Manutenção e Execução dos Testes

A manutenção regular dos testes é crucial, pois à medida que a aplicação evolui, os testes podem se tornar desatualizados ou quebrar devido a mudanças no código. Portanto, **refatore os testes** quando necessário, e mantenha os dados de teste atualizados.

Além disso, execute os testes automaticamente em **todas as alterações** de código e **pull requests** para garantir que a base de código permaneça **estável**.

8. Boas Práticas no Desenvolvimento de Testes Automatizados

- **Manter Testes Independentes**: Cada teste deve ser independente, ou seja, não depender de outros testes para ser executado.
- **Utilizar Dados de Teste Isolados**: Evite usar dados compartilhados entre testes, o que pode causar efeitos colaterais.
- **Testar o Cenário Completo**: Garanta que os testes cubram cenários reais de uso e não apenas casos isolados.
- **Execução Rápida**: Priorize a criação de testes rápidos e que possam ser executados frequentemente sem consumir muito tempo.
- **Feedback Rápido**: Os testes devem fornecer **feedback rápido** para a equipe, permitindo que as falhas sejam corrigidas imediatamente.

Execução de Testes Automatizados

A **execução de testes automatizados** é uma das etapas mais importantes no

ciclo de vida dos testes de software. Ela garante que as funcionalidades do sistema estejam funcionando corretamente e ajuda a detectar problemas antes que cheguem ao usuário final. A execução dos testes pode ser feita localmente, em servidores de integração contínua (CI), ou em nuvens de testes distribuídos. Vamos explorar o processo de execução de testes, melhores práticas e ferramentas utilizadas.

1. Preparação para a Execução dos Testes

Antes de executar os testes, alguns preparativos são necessários:

Configuração do Ambiente de Testes

O ambiente deve ser configurado de forma a refletir o ambiente de produção ou o cenário de teste que você deseja validar. A configuração pode incluir:

- **Instalação de dependências** (ex: frameworks de automação, bibliotecas de teste).
- **Configuração de variáveis de ambiente** (ex: URL de APIs, credenciais de acesso).
- **Banco de dados**: Se necessário, crie ou use um banco de dados de teste isolado.
- **Inicialização de sistemas e servidores**: Caso haja a necessidade de simular serviços externos ou iniciar servidores locais.

Dados de Teste

Garanta que os **dados de teste** estejam preparados. Isso pode incluir dados em banco de dados ou arquivos (ex: CSV, JSON). Dependendo do tipo de teste, você pode usar dados **dinâmicos** ou **mockar** APIs.

Testes de Configuração

Antes de rodar os testes, é importante garantir que o sistema esteja na configuração certa:

- **WebDriver** (para testes de UI) está configurado corretamente (seja Selenium, Cypress, etc.).
- Dependências do **framework de testes** estão instaladas (como JUnit, TestNG, Mocha).
- A **rede de comunicação** (para APIs) está funcional.

2. Execução Local dos Testes

Testes de UI (Selenium, Cypress, Playwright)

- Para **testes de UI**, a execução geralmente envolve a **abertura do navegador** e

a interação com a interface. Você pode rodar os testes localmente, no seu computador, utilizando um navegador (Chrome, Firefox, etc.) ou até mesmo um navegador sem cabeça (headless) para execução mais rápida e sem interface gráfica.

Testes de API (Postman, RestAssured)

- Para **testes de API**, os testes podem ser executados através de **scripts de requisições HTTP** que enviam e recebem dados de uma API. O Postman, por exemplo, permite executar as requisições diretamente do seu ambiente de desenvolvimento.
- No **Postman**, você pode usar a opção de **Runner** para rodar os testes em lote ou configurar uma coleção de testes automatizados. No **RestAssured** (Java), a execução pode ser realizada diretamente a partir de um IDE como o IntelliJ IDEA ou Eclipse.

Testes Unitários

- Para testes unitários com frameworks como **JUnit** ou **TestNG**, você pode rodar os testes diretamente a partir da IDE, como o **Eclipse** ou **IntelliJ**, ou usar ferramentas de linha de comando, como o **Maven** ou **Gradle**.

3. Execução em Integração Contínua (CI)

A execução de testes em um ambiente de **Integração Contínua (CI)** é fundamental para garantir que os testes sejam executados automaticamente a cada nova alteração no código. Isso ajuda a garantir a **qualidade contínua** e permite detectar falhas rapidamente.

Configuração de CI (Ex: Jenkins, GitLab CI, Travis CI)

- A configuração de uma ferramenta de CI envolve a criação de um pipeline que automatiza a execução dos testes. Quando um **commit** é feito no repositório, a ferramenta de CI inicia o processo de **build** e **execução dos testes**.
- O **Jenkins** e outras ferramentas CI têm funcionalidades de integração com **Slack** ou **e-mail** para notificar sobre o status da execução dos testes (passou ou falhou).

4. Execução Distribuída e Testes em Nuvem

A execução de testes pode ser feita de forma **distribuída** ou na **nuvem** para aproveitar a capacidade de vários **nós** (máquinas virtuais) ou **infraestrutura escalável**.

Ferramentas para Execução Distribuída

- **Selenium Grid**: Permite executar testes de Selenium em múltiplos navegadores e plataformas ao mesmo tempo.
- **BrowserStack** e **Sauce Labs**: Ferramentas na nuvem para executar testes cross-browser e em dispositivos móveis.

Execução Paralela

A execução de testes de forma **paralela** ajuda a reduzir o tempo de execução e a aumentar a cobertura dos testes. Isso pode ser configurado diretamente nas ferramentas de CI, usando **nós paralelos**.

5. Analisando os Resultados da Execução dos Testes

Após a execução dos testes, a análise dos resultados é uma etapa crucial. Algumas das formas de análise incluem:

Relatórios de Testes

Ferramentas como **Allure Report**, **ExtentReports** e **JUnit** geram **relatórios visuais** com o status de cada teste, facilitando a análise de falhas e erros.

Logs de Execução

A análise de **logs** durante a execução dos testes fornece informações detalhadas sobre o que aconteceu durante o processo. Isso é especialmente importante para depurar falhas.

Testes com Falhas

Identifique testes que falharam e analise a causa da falha:

- Verifique se a falha foi causada por uma **alteração recente no código** ou um problema no **ambiente** de testes.
- Se necessário, revise o **caso de teste** para garantir que ele esteja validando a funcionalidade corretamente.

6. Boas Práticas na Execução dos Testes

- **Execução Frequente**: Execute os testes frequentemente para detectar falhas o mais cedo possível.
- **Automação da Execução**: Utilize **CI/CD** para automatizar a execução dos testes sempre que uma alteração é feita no código.
- **Cobertura Abrangente**: Garanta que seus testes cubram todas as partes críticas do sistema, incluindo fluxos de usuário importantes e interações entre componentes.
- **Manutenção de Testes**: À medida que o código da aplicação muda, os testes

também precisam ser **atualizados** para refletir essas mudanças.

- **Execução Paralela:** Execute os testes em **múltiplos ambientes** e **navegadores** simultaneamente para aumentar a cobertura e reduzir o tempo de execução.

GitHub da Disciplina

Confira os códigos e scripts usados ao longo da disciplina no repositório dela no GitHub: <https://github.com/FaculdadeDescomplica/Pratica-Integradora-com-Metodos-Ageis>

Conteúdo Bônus

Um documentário cujo foco é a diversidade, aborda alguns trechos sobre práticas modernas de desenvolvimento, testes automatizados e TDD é o “Code: Debugging the Gender Gap” de 2015; um curta documental que mostra sobre comunidade de testes de software, boas práticas, incluindo automação é o “The testing planet”. Outro documentário indicado é o “Quality for DevOps Teams” que integra automação de testes com DevOps e qualidade de software.

Referências Bibliográficas

MESZAROS, Gerard. xUnit Test Patterns: Refactoring Test Code. Boston: Addison-Wesley, 2007.

BECK, Kent. Test-Driven Development: By Example. Boston: Addison-Wesley, 2003.

FOWLER, Martin. The Test Pyramid. [S. l.]: martinfowler.com, 2012. Disponível em: <https://martinfowler.com/bliki/TestPyramid.html>. Acesso em: 25 abr. 2025.

GLOVER, David; FAY, Stuart. Continuous Testing for DevOps Professionals. Sebastopol: O'Reilly Media, 2017.

10 ferramentas para teste automatizado de software. Malu, 16 de janeiro de 2025. Disponível em <https://uds.com.br/blog/ferramentas-para-teste-automatizado-de-software/>. Acesso em 24 de abril de 2025.

Título da Prática: Automação de testes na prática

Objetivos: Exercitar conceitos, na prática, de testes de aplicações

Materiais, Métodos e Ferramentas: Para realizar esta prática vamos ler atentamente o conteúdo da unidade

Atividade Prática

Sua equipe de desenvolvimento está implementando um processo de testes automatizados para um sistema de vendas online. O objetivo é garantir que todas as funcionalidades críticas, como cadastro de produtos, processamento de pagamentos e envio de notificações, sejam testadas automaticamente a cada nova alteração no código. Como parte desse processo, é necessário configurar o ambiente, estruturar e desenvolver os testes, bem como realizar sua execução de forma eficiente.

1. Explique a importância de configurar o ambiente antes de iniciar a automação de testes e cite dois elementos essenciais dessa configuração no contexto do sistema de vendas.
2. Descreva como deve ser feita a estruturação de testes automatizados e explique por que essa etapa é fundamental.
3. Explique o que envolve o desenvolvimento de testes automatizados e cite um exemplo aplicado ao sistema de vendas online.
4. Detalhe o que significa a execução dos testes automatizados e explique um benefício de automatizar essa etapa no processo de desenvolvimento.

Gabarito Atividade Prática

1. Configurar o ambiente é importante para garantir que os testes automatizados sejam executados de maneira consistente e controlada. No sistema de vendas online, dois elementos essenciais dessa configuração são:
A instalação das bibliotecas de automação, como Selenium ou Cypress.
A definição de variáveis de ambiente, como URLs de testes e credenciais fictícias.
2. A estruturação de testes automatizados consiste em organizar o código de testes de forma lógica e modular, facilitando a manutenção e a escalabilidade. Isso inclui separar testes por tipo (unitários, integração, end-to-end), criar pastas específicas e padronizar nomes. Essa etapa é fundamental para evitar redundâncias, facilitar a leitura e permitir que novos membros da equipe compreendam rapidamente a organização dos testes.
3. O desenvolvimento de testes automatizados envolve escrever o código que simula a interação com o sistema e verifica se o comportamento esperado ocorre. Por exemplo, no sistema de vendas online, um teste automatizado pode verificar se, ao adicionar um produto ao carrinho e finalizar a compra, o sistema gera uma confirmação de pagamento.

4. A execução dos testes automatizados significa rodar os scripts criados para verificar se o sistema está funcionando conforme o esperado. Um benefício de automatizar essa etapa é a possibilidade de realizar testes continuamente (por exemplo, com integração contínua), garantindo a detecção rápida de falhas sempre que novas alterações são feitas no código.